PTC-BIAS: PHONEME-LEVEL TEMPORAL COMPETITION FOR BIAS RETRIEVAL AND POST-DECODING CORRECTION IN SPEECH LLMS

Zhiqi Ai[1], Han Cheng[1], Shiyi Mu[1], Yongjin Zhou[1,], Shugong Xu[2,*]*

[1]Shanghai University, Shanghai, China
[2]Xi'an Jiaotong-Liverpool University, Suzhou, China

arXiv:2609.28727v1 [cs.CL] 23 Sep 2026

ABSTRACT

Contextual biasing improves rare-word recognition in speech large language models (SpeechLLMs), but efficiently exploiting large bias lists remains challenging. We propose **PTC-Bias**, a two-stage framework based on phoneme-level temporal competition. At the prefill stage, **PTC Retrieval** performs frame-synchronous phoneme decoding and temporal competition among candidate pronunciations, producing a compact bias-word shortlist and corresponding speech intervals. After SpeechLLM decoding, **PTC Correction** conducts a second local competition between the retrieved candidates and mismatched transcript spans within these intervals. Selective correction reduces near-homophone and word-segmentation errors while preserving correct transcriptions. Both stages share the same phoneme posteriors and require no additional SpeechLLM forward pass. Experiments on LibriSpeech show consistent gains across two SpeechLLMs and bias lists of up to 2000 words. With Prompt-SLAM-ASR-7B and 2000 bias words, PTC-Bias reduces B-WER by 23.4%/23.9% relative to CTC-Filter on test-clean/test-other, while keeping U-WER nearly unchanged.

***Index Terms*—** speech large language models, contextual biasing, phoneme CTC, temporal competition, post-decoding correction

1. INTRODUCTION

Speech large language models (SpeechLLMs) have achieved strong performance in automatic speech recognition (ASR), but they still struggle with rare words such as names, locations, and technical terms. Contextual ASR addresses this problem by incorporating a user-provided bias list through deep biasing, text-based adaptation, or contextual prompting [1–4]. For SpeechLLMs, however, directly inserting a large bias list into the prompt increases inference cost and may introduce lexical interference, although only a small fraction of the list is usually relevant to a given utterance. This has motivated retrieval-based approaches that select a compact set of acoustically relevant bias words before SpeechLLM decoding [5–7].

Existing bias-word selection methods rely on phonetic similarity, speech–text representations, intermediate ASR hypotheses, or learned retrieval scores [5–10]. Embedding-based methods scale efficiently to large bias lists but often lack precise temporal localization, whereas hypothesis-based methods may miss a relevant word if it is absent from the initial transcription. Moreover, most methods rank candidates independently, allowing multiple near-homophones associated with the same speech segment to be selected together rather than compared directly.

Phoneme-based keyword spotting offers finer frame-level evidence. CTC-based word spotters search candidate pronunciations over phoneme posteriors [11, 12], while recent user-defined keyword-spotting systems employ streaming phoneme search, multi-stage matching, or frame-asynchronous decoding [13–15]. However, these methods are primarily designed to detect individual keywords and typically evaluate each candidate through a separate decoding or matching path, without explicitly modeling competition between overlapping pronunciations.

Retrieval alone is also insufficient for reliable contextual correction. Even when the correct bias word is included in the prompt, a SpeechLLM may still output a frequent near-homophone or produce an incorrect word segmentation. Existing word-spotting approaches can insert detected keywords into ASR hypotheses [11, 16], but a false detection may lead to an unnecessary replacement of an already correct transcription. Reliable correction therefore requires a second comparison between the retrieved bias word and the corresponding transcript span, using acoustic evidence localized to the detected speech interval.

To address these limitations, we propose **PTC-Bias**[1], a two-stage contextual biasing framework based on phoneme-level temporal competition. During prefill, **PTC Retrieval** performs frame-synchronous phoneme decoding and temporal competition among candidate pronunciations, producing a compact bias-word shortlist together with their speech intervals. After SpeechLLM decoding, **PTC Correction** reuses the detected intervals and cached phoneme posteriors to conduct a second local competition between each retrieved candidate and its mismatched transcript span. The two stages thus use the same phoneme-level acoustic evidence and require no additional SpeechLLM forward pass. Experiments on LibriSpeech demonstrate consistent improvements over the corresponding SpeechLLM and CTC-Filter baselines across two SpeechLLMs and bias-list sizes ranging from 100 to 2000. With Prompt-SLAM-ASR-7B and 2000 bias words, PTC-Bias reduces B-WER by 23.4% and 23.9% relative to CTC-Filter on test-clean and test-other, respectively, while keeping U-WER nearly unchanged.

2. PROPOSED METHOD

2.1. Overview

As shown in Fig. 1, PTC-Bias uses a lightweight phoneme-CTC branch to obtain frame-level phoneme posteriors. Before decoding, PTC Retrieval selects bias words and their speech intervals via temporal competition for prompting; after decoding, PTC Correction uses the same posteriors to correct mismatched transcript spans within those intervals. Both stages share the phoneme posteriors and require no additional audio-encoder or SpeechLLM forward pass.

*Corresponding authors

[1]Project code: https://github.com/aizhiqi-work/PTC-Bias

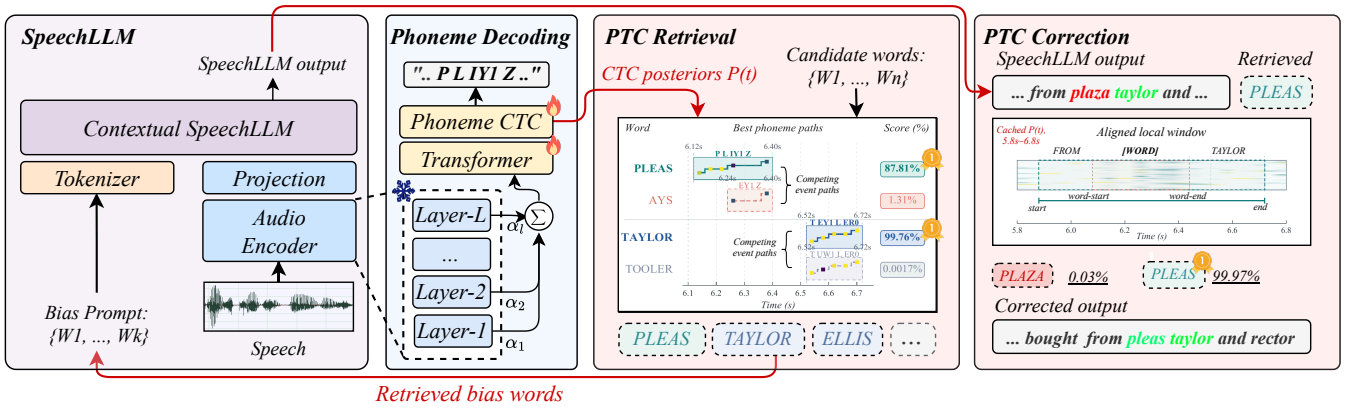


Fig. 1. Overview of PTC-Bias. A lightweight phoneme-CTC branch extracts frame-level phoneme posteriors from the frozen audio encoder. PTC Retrieval performs temporal competition to select acoustically supported bias words and locate their speech intervals before SpeechLLM decoding. PTC Correction then reuses the intervals and cached posteriors to correct mismatched transcript spans through local competition.

2.2. Contextual SpeechLLM

The Contextual SpeechLLM comprises an audio encoder, an audio projection module, and an autoregressive LLM decoder. The projected speech representations and a textual bias prompt are jointly provided to the LLM for transcription. Since a large bias list increases the prompt length and may introduce lexical interference [7], we include at most K words selected by PTC Retrieval (Sec. 2.4) and retain their speech intervals for subsequent verification. This preserves the original SpeechLLM interface without modifying its parameters. Remaining errors are handled by PTC Correction (Sec. 2.5).

2.3. Phoneme Decoding

We attach a lightweight phoneme-CTC branch to the frozen audio encoder. Its input is a learned weighted sum of all L Transformer-layer outputs:

$$\overline{\mathbf{H}}_t = \sum_{\ell=1}^{L} \alpha_\ell \mathbf{H}_t^{(\ell)}, \qquad \boldsymbol{\alpha} = \mathrm{softmax}(\mathbf{g}), \tag{1}$$

where $\mathbf{H}_t^{(\ell)}$ is the representation at frame t from layer ℓ, and α_ℓ is its normalized weight. A lightweight Transformer and phoneme-CTC head then produce frame-level phoneme posteriors. Reference transcripts and bias words are converted into stress-marked ARPAbet sequences using g2pE [17], providing a shared phoneme vocabulary for training and retrieval. Only the fusion weights, lightweight Transformer, and CTC head are optimized. The resulting posteriors are computed once and shared by PTC Retrieval and PTC Correction.

2.4. PTC Retrieval

PTC Retrieval converts the bias words into stress-marked phoneme sequences and compiles them into a shared prefix trie. Following CTC-based word spotting and streaming phoneme search [11,12,14, 15], it searches the phoneme posteriors frame by frame. Each candidate may start at any frame, while candidates sharing a phoneme prefix reuse the same decoding states. Algorithm 1 summarizes the procedure.

Algorithm 1: PTC Retrieval

Input: Phoneme posteriors $\mathbf{P}_{1:T}$; bias list $\mathcal{B}$
Output: Retrieved events $\mathcal{R}$
$\mathcal{W} \leftarrow \mathrm{G2P}(\mathcal{B})$;
$\mathcal{G} \leftarrow \textsc{BuildTrie}(\mathcal{W})$;
$\mathcal{D} \leftarrow \textsc{InitializeCTC}(\mathcal{G})$;
$\mathcal{E} \leftarrow \varnothing$;
for $t \leftarrow 1$ **to** T **do**
  $\mathcal{D} \leftarrow \textsc{TrieCTCStep}(\mathcal{D}, \mathbf{P}_t)$;
  $\mathcal{E} \leftarrow \textsc{UpdateBestEvents}(\mathcal{E}, \mathcal{D}, t, \beta)$;
$\mathcal{E}_M \leftarrow \textsc{TopM}(\mathcal{E}, M)$;
$\{q_k\} \leftarrow \textsc{IntervalFB}(\mathcal{E}_M, \rho, \tau)$;
$\mathcal{R} \leftarrow \textsc{TopKByMarginal}\big(\{(\xi_k, q_k) : \xi_k \in \mathcal{E}_M,\ q_k \geq \eta\}, K\big)$;
return $\mathcal{R}$;

For a bias word v_k with pronunciation $\mathbf{w}^{(k)}$ of length U_k, the search retains its best local CTC path and corresponding interval $[a_k, b_k]$. The length-calibrated score is

$$S_k = \max_{\substack{1 \le a \le b \le T \\ \mathcal{C}(\boldsymbol{\pi}_{a:b}) = \mathbf{w}^{(k)}}} \left[\frac{1}{U_k} \sum_{t=a}^{b} \log P_t(\pi_t) + \beta \log U_k \right], \tag{2}$$

where $\mathcal{C}$ is the CTC collapse operator, $P_t(\pi_t)$ is the posterior probability of CTC symbol π_t at frame t, and β controls length calibration. The maximizing path determines the interval $[a_k, b_k]$.

Each bias word contributes its best event $\xi_k = (v_k, [a_k, b_k], S_k)$. We retain the M highest-scoring events as $\mathcal{E}_M$ and treat events with overlapping intervals as competing explanations of the same acoustic evidence. Let $\mathfrak{C}_M$ denote all subsets of $\mathcal{E}_M$ whose events have non-overlapping intervals. The empty subset is included to allow no bias word to be selected. The weight of a compatible event set $\mathcal{A} \in \mathfrak{C}_M$ is

$$\psi(\mathcal{A}) = \exp\left(\sum_{\xi_j \in \mathcal{A}} \frac{S_j - \rho}{\tau} \right), \tag{3}$$

where ρ is the per-event reference score and τ is the competition temperature. The competition-adjusted confidence of event ξ_k is its

Table 1. Contextual ASR results (%) on LibriSpeech with different numbers of distractors. Each entry reports test-clean / test-other. In the Qwen3-ASR block, † denotes WavLM-based phoneme posteriors; otherwise, the AuT front-end is used. Lower is better.

Contextual ASR Model	$N = 100$			$N = 500$			$N = 1000$			$N = 2000$		
	WER	B-WER	U-WER	WER	B-WER	U-WER	WER	B-WER	U-WER	WER	B-WER	U-WER
DB-NNLM [1]	1.98 / 5.86	5.70 / 14.10	1.50 / 4.90	2.09 / 6.09	6.20 / 15.10	1.60 / 5.10	2.14 / 6.35	6.70 / 17.20	1.60 / 5.10	2.27 / 6.58	7.30 / 18.90	1.60 / 5.20
USTR-CT [3]	2.06 / 5.38	2.00 / 4.40	2.10 / 5.50	2.09 / 5.62	2.20 / 5.60	2.10 / 5.60	2.16 / 5.75	2.50 / 6.30	2.10 / 5.70	2.17 / 5.84	3.00 / 7.60	2.10 / 5.60
CB-QwenAudio [4]	1.60 / 3.80	5.50 / 13.50	1.30 / 2.60	1.90 / 3.90	6.00 / 14.20	1.40 / 2.70	–	–	–	–	–	–
Prompt-SLAM-ASR-7B [5]	7.40 / 17.90	24.80 / 44.10	–	–	–	–	–	–	–	–	–	–
+ CTC-Filter [5]	1.27 / 2.72	3.67 / 8.02	1.00 / 2.16	1.33 / 3.04	3.92 / 9.04	1.03 / 2.40	1.33 / 2.99	4.16 / 9.33	1.00 / 2.31	1.38 / 3.20	4.41 / 10.02	1.03 / 2.47
+ Bias Retrieval [7]	1.30 / 2.70	3.80 / 8.30	–	1.30 / 3.00	4.00 / 8.70	–	1.30 / 3.10	4.10 / 9.00	–	1.40 / 3.10	4.20 / 9.30	–
+ PTC Retrieval	1.19 / 2.48	3.30 / 6.76	0.95 / 2.02	1.25 / 2.64	3.56 / 7.08	0.99 / 2.16	1.33 / 2.72	3.58 / 7.41	1.06 / 2.22	1.36 / 3.00	3.85 / 8.24	1.07 / 2.44
+ PTC Correction	1.13 / 2.41	2.76 / 6.16	0.94 / 2.01	1.19 / 2.58	3.01 / 6.51	0.98 / 2.16	1.26 / 2.67	3.01 / 6.90	1.05 / 2.22	1.30 / 2.93	3.38 / 7.63	1.06 / 2.43
Qwen3-ASR-0.6B [18]	1.87 / 4.12	7.14 / 15.04	1.26 / 2.95	2.12 / 4.57	8.81 / 18.78	1.35 / 3.06	–	–	–	–	–	–
+ PTC Retrieval	1.48 / 3.16	4.25 / 7.80	1.16 / 2.67	1.51 / 3.30	4.38 / 8.63	1.18 / 2.73	1.52 / 3.32	4.54 / 8.76	1.17 / 2.74	1.60 / 3.42	5.08 / 9.24	1.20 / 2.80
+ PTC Correction	1.41 / 3.07	3.63 / 7.00	1.15 / 2.65	1.44 / 3.21	3.70 / 7.73	1.18 / 2.72	1.46 / 3.26	3.94 / 8.04	1.18 / 2.75	1.54 / 3.37	4.45 / 8.61	1.21 / 2.81
+ CTC-Filter† [5]	1.51 / 3.25	4.39 / 8.35	1.18 / 2.71	1.53 / 3.33	4.52 / 9.14	1.18 / 2.71	1.56 / 3.40	4.79 / 9.53	1.18 / 2.74	1.57 / 3.41	4.81 / 9.45	1.19 / 2.77
+ PTC Retrieval†	1.47 / 3.13	4.23 / 7.65	1.14 / 2.64	1.50 / 3.21	4.30 / 8.14	1.18 / 2.68	1.51 / 3.21	4.41 / 8.16	1.17 / 2.68	1.56 / 3.26	4.88 / 8.53	1.18 / 2.70
+ PTC Correction†	1.38 / 3.03	3.47 / 6.84	1.14 / 2.62	1.41 / 3.10	3.54 / 7.24	1.17 / 2.66	1.42 / 3.11	3.67 / 7.33	1.16 / 2.66	1.47 / 3.17	4.10 / 7.76	1.17 / 2.68

inclusion marginal

$$q_k = \frac{\sum_{\substack{\mathcal{A} \in \mathfrak{C}_M \\ \xi_k \in \mathcal{A}}} \psi(\mathcal{A})}{\sum_{\mathcal{A} \in \mathfrak{C}_M} \psi(\mathcal{A})}. \tag{4}$$

These marginals are computed efficiently using interval forward–backward. Events satisfying $q_k \geq \eta$ are ranked by q_k, and at most K bias words, together with their intervals and retrieval confidences, are passed to the Contextual SpeechLLM.

2.5. PTC Correction

A retrieved bias word may still be transcribed as a near-homophone or split into multiple words. PTC Correction aligns the initial transcript $\hat{\mathbf{y}}$ to the cached phoneme posteriors and identifies a short, phonetically similar span $\hat{\mathbf{y}}_{i:j}$ overlapping each retrieved interval $[a_k, b_k]$. To compare the retrieved word v_k with this span under the same phonetic context, we form $\mathbf{z}_k^+ = \mathbf{c}_L \oplus \mathrm{G2P}(v_k) \oplus \mathbf{c}_R$ and $\mathbf{z}_k^- = \mathbf{c}_L \oplus \mathrm{G2P}(\hat{\mathbf{y}}_{i:j}) \oplus \mathbf{c}_R$, where $\mathbf{c}_L$ and $\mathbf{c}_R$ are shared context phonemes.

Both sequences are scored within a local window Ω_k extended by five posterior frames on each side. Their acoustic margin is

$$\Delta_k = F(\mathbf{z}_k^+; \Omega_k) - F(\mathbf{z}_k^-; \Omega_k), \tag{5}$$

where F denotes the CTC log forward probability. A replacement requires Δ_k to exceed a threshold and pass lexical and boundary checks. Exact homophones, which phoneme-CTC cannot distinguish, are handled only under restricted rare-word or segmentation conditions. Conflicting edits are resolved using Δ_k and the retrieval marginal q_k from Eq. (4). All comparisons reuse cached posteriors without an additional audio-encoder or SpeechLLM forward pass.

3. EXPERIMENTAL SETUP

3.1. Datasets and Protocol

We train the phoneme-CTC front-ends on the 460-hour clean subset and the full 960-hour training set of LibriSpeech [19]. Evaluation is conducted on test-clean and test-other, containing 2,620 and 2,939 utterances, respectively. Following the Rare5k protocol [1, 7], each utterance is paired with its oracle rare words and $N \in \{100, 500, 1000, 2000\}$ distractors. PTC Retrieval selects at most ten words from this list and supplies them as a textual prompt to Prompt-SLAM-ASR-7B [5] or Qwen3-ASR-0.6B [18]. Both SpeechLLM backbones are kept frozen.

3.2. Models and Implementation

We compare DS-KWS [13, 15], WavLM [20], and Qwen3-ASR's audio Transformer (AuT) [18] as phoneme front-ends. DS-KWS is a 3.61M-parameter KWS baseline operating at 25 Hz. For WavLM and AuT, we freeze the encoders and train only a layer mixture, a two-layer Transformer, and a 71-class phoneme-CTC head. WavLM operates at 50 Hz with 317.31M total and 1.86M trainable parameters, while AuT operates at 12.5 Hz with 186.48M total and 1.83M trainable parameters. Training uses WeNet [21] with a batch size of 8 on four NVIDIA RTX 3090 GPUs.

PTC Retrieval uses $M = 100$, $K = 10$, $\beta = 1.1$, $\rho = 0$, $\tau = 0.25$, and $\eta = 0.05$. PTC Correction extends each local window by five posterior frames on both sides and applies an acoustic-margin threshold of 2.0. Both stages share the cached phoneme posteriors.

3.3. Evaluation Metrics

We report phoneme error rate (PER), overall word error rate (WER), biased-word error rate (B-WER), and unbiased-word error rate (U-WER). Following BR-ASR [7], retrieval is evaluated using $\mathrm{Recall}_B@99$, the mean number of candidates required to reach 99% oracle-word recall; $\mathrm{Recall}_B\#50$, the oracle-word recall within the top 50; and $\mathrm{Recall}_H\#50$, the near-homophone distractor recall within the top 50.

4. EXPERIMENTAL RESULTS

4.1. Main Results

Table 1 compares PTC-Bias with DB-NNLM [1], USTR-CT [3], CB-QwenAudio [4], CTC-Filter [5], and Bias Retrieval [7]. At $N = 2000$, PTC-Bias with Prompt-SLAM-ASR-7B achieves WERs of 1.30%/2.93% and B-WERs of 3.38%/7.63% on test-clean/test-other. Compared with CTC-Filter, it reduces B-WER

Table 2. Phoneme error rates (PER, %) of different phoneme front-ends on LibriSpeech. Lower is better.

Front-end	LS-460		LS-960	
	test-clean	test-other	test-clean	test-other
DS-KWS [13, 15]	4.44	13.39	4.45	11.80
AuT [18]	2.39	5.88	2.06	5.13
WavLM [20]	**1.29**	**2.44**	**1.13**	**2.27**

Table 3. Bias retrieval on LibriSpeech test-other with $N = 2000$ distractors. Acoustic and Textual are BR-ASR [7] variants .

Method	Recall_B@99 ↓	Recall_B#50 (%) ↑	Recall_H#50 (%) ↓
BR-ASR (Acoustic) [7]	42.2	**99.7**	69.3
BR-ASR (Textual) [7]	47.3	99.1	58.4
PTC Retrieval	**16.9**	99.3	**22.0**

by 23.4%/23.9% while keeping U-WER nearly unchanged. With Qwen3-ASR-0.6B and its AuT phoneme front-end [18], PTC-Bias achieves WERs of 1.54%/3.37% and B-WERs of 4.45%/8.61%. The consistent gains from $N = 100$ to $N = 2000$ demonstrate its robustness to large bias lists.

The nested PTC Correction rows further show the benefit of post-decoding competition. At $N = 2000$, it reduces B-WER from 3.85%/8.24% to 3.38%/7.63% with Prompt-SLAM-ASR-7B, and from 5.08%/9.24% to 4.45%/8.61% with Qwen3-ASR. This indicates that retrieving the correct bias word does not always guarantee correct generation, while local competition can further resolve near-homophone confusions and span-boundary errors. The † rows reuse the WavLM [20] retrieval results from Prompt-SLAM-ASR for Qwen3-ASR decoding; under this controlled setting, PTC-Bias reduces B-WER by 14.8%/17.9% relative to CTC-Filter.

4.2. Phoneme Modeling and Bias Retrieval

Table 2 compares the phoneme front-ends, using DS-KWS [13, 15] as a lightweight KWS-oriented baseline. With LS-960 training, WavLM achieves PERs of 1.13%/2.27% on test-clean/test-other, compared with 2.06%/5.13% for AuT and 4.45%/11.80% for DS-KWS. In particular, the AuT results indicate that existing Speech-LLM audio encoders provide sufficient phoneme discrimination to support both PTC stages.

Table 3 further compares PTC Retrieval with BR-ASR [7]. Relative to acoustic BR-ASR, PTC Retrieval reduces Recall_B@99 from 42.2 to 16.9 and Recall_H#50 from 69.3% to 22.0%, while maintaining Recall_B#50 at 99.3%. PTC Retrieval therefore reaches high oracle-word recall with a smaller shortlist and substantially reduces near-homophone interference.

4.3. Qualitative and Efficiency Analysis

Figure 2 visualizes phoneme-level temporal competition in PTC Retrieval. Similar pronunciations often produce overlapping high-scoring paths over the same speech segment. By treating these paths as competing explanations, PTC Retrieval suppresses weaker near-homophones while retaining confident events at distinct intervals, producing a compact and less ambiguous shortlist.

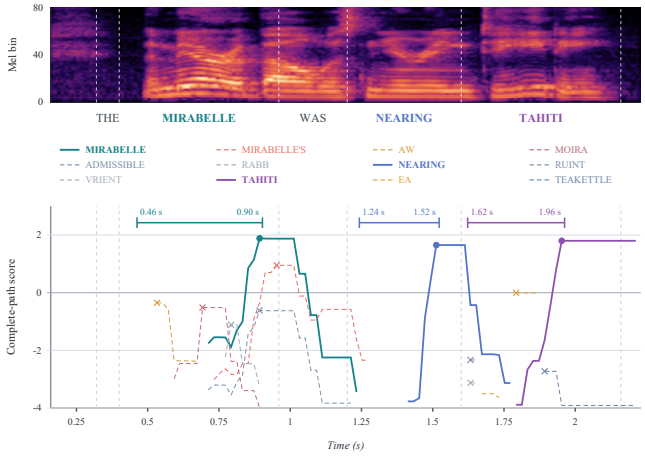


Fig. 2. Temporal competition among phoneme paths in PTC Retrieval. Solid paths are selected; dashed paths are competing alternatives.

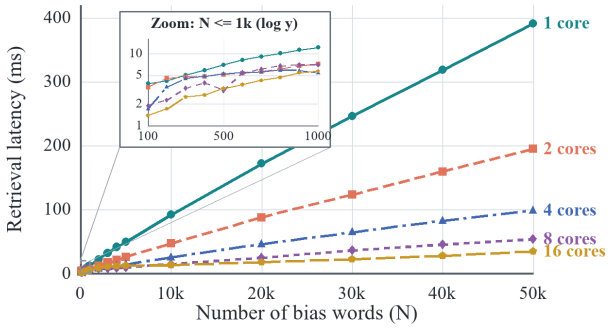


Fig. 3. CPU search latency of PTC Retrieval for a 5.09-s utterance across bias-list sizes and worker counts. Phoneme-posterior extraction and trie construction are excluded.

Figure 3 reports CPU retrieval latency for a 5.09-s utterance, excluding phoneme-posterior computation and trie construction. At $N = 2000$, retrieval takes 7.61 ms with four workers. For $N = 50{,}000$, increasing the worker count from one to 16 reduces latency from 391.24 ms to 34.74 ms. These results show that CPU parallelism enables PTC Retrieval to scale efficiently to large bias lists.

5. CONCLUSION

We presented PTC-Bias, a two-stage contextual biasing framework for SpeechLLMs based on phoneme-level temporal competition. Before decoding, PTC Retrieval uses frame-level phoneme posteriors to select bias words and locate their speech intervals; after decoding, PTC Correction compares misrecognized transcript spans with the retrieved candidates. Both stages reuse the same phoneme posteriors without an additional SpeechLLM forward pass. Experiments on LibriSpeech show consistent improvements with 100–2000 distractors, different phoneme front-ends, and two SpeechLLM backbones. PTC-Bias substantially reduces B-WER while largely preserving U-WER and maintains efficient CPU retrieval for large bias lists.

6. REFERENCES

[1] Duc Le, Mahaveer Jain, Gil Keren, Suyoun Kim, Yangyang Shi, Jay Mahadeokar, Julian Chan, Yuan Shangguan, Christian Fuegen, Ozlem Kalinli, Yatharth Saraf, and Michael L. Seltzer, "Contextualized Streaming End-to-End Speech Recognition with Trie-Based Deep Biasing and Shallow Fusion," in *Proceedings of Interspeech*, 2021, pp. 1772–1776.

[2] Lu Huang, Boyu Li, Jun Zhang, Lu Lu, and Zejun Ma, "Text-only Domain Adaptation Using Unified Speech-Text Representation in Transducer," in *Proceedings of Interspeech*, 2023, pp. 386–390.

[3] Jin Qiu, Lu Huang, Boyu Li, Jun Zhang, Lu Lu, and Zejun Ma, "Improving Large-Scale Deep Biasing With Phoneme Features and Text-Only Data in Streaming Transducer," in *2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023, pp. 1–8.

[4] Xun Gong, Anqi Lv, Zhiming Wang, and Yanmin Qian, "Contextual Biasing Speech Recognition in Speech-Enhanced Large Language Model," in *Proceedings of Interspeech*, 2024, pp. 257–261.

[5] Guanrou Yang, Ziyang Ma, Zhifu Gao, Shiliang Zhang, and Xie Chen, "CTC-Assisted LLM-Based Contextual ASR," in *2024 IEEE Spoken Language Technology Workshop (SLT)*, 2024, pp. 126–131.

[6] Zhihong Lei, Xingyu Na, Mingbin Xu, Ernest Pusateri, Christophe Van Gysel, Yuanyuan Zhang, Shiyi Han, and Zhen Huang, "Contextualization of ASR with LLM Using Phonetic Retrieval-Based Augmentation," in *2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.

[7] Xun Gong, Anqi Lv, Wangyou Zhang, Zhiming Wang, Huijia Zhu, and Yanmin Qian, "BR-ASR: A Unified Framework for Contextualized Speech Recognition with LLM-Based Retrieval and Reasoning," in *Proceedings of Interspeech*, 2025, pp. 4043–4047.

[8] Haoxiang Hou, Xun Gong, Wangyou Zhang, Wei Wang, and Yanmin Qian, "Ranking and Selection of Bias Words for Contextual Bias Speech Recognition," in *Proceedings of Interspeech*, 2025, pp. 5183–5187.

[9] Zhiqi Huang, Diamantino Caseiro, Kandarp Joshi, Christopher Li, Pat Rondon, Zelin Wu, Petr Zadrazil, and Lillian Zhou, "Optimizing Large-Scale Context Retrieval for End-to-End ASR," in *Proceedings of Interspeech*, 2024, pp. 4573–4577.

[10] Nikolaos Flemotomos, Roger Hsiao, Pawel Swietojanski, Takaaki Hori, Dogan Can, and Xiaodan Zhuang, "Optimizing Contextual Speech Recognition Using Vector Quantization for Efficient Retrieval," *IEEE Transactions on Audio, Speech and Language Processing*, vol. 33, pp. 3508–3521, 2025.

[11] Andrei Andrusenko, Aleksandr Laptev, Vladimir Bataev, Vitaly Lavrukhin, and Boris Ginsburg, "Fast Context-Biasing for CTC and Transducer ASR Models with CTC-Based Word Spotter," in *Proceedings of Interspeech*, 2024, pp. 757–761.

[12] Yu Nakagome and Michael Hentschel, "WCTC-Biasing: Retraining-Free Contextual Biasing ASR with Wildcard CTC-Based Keyword Spotting and Inter-Layer Biasing," in *Proceedings of Interspeech*, 2025, pp. 5178–5182.

[13] Zhiqi Ai, Han Cheng, Yuxin Wang, Shiyi Mu, Yongjin Zhou, and Shugong Xu, "Dual Data Scaling for Robust Two-Stage User-Defined Keyword Spotting," in *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 18307–18311.

[14] Yu Xi, Haoyu Li, Xiaoyu Gu, Yidi Jiang, and Kai Yu, "MFA-KWS: Effective Keyword Spotting With Multi-Head Frame-Asynchronous Decoding," *IEEE Transactions on Audio, Speech and Language Processing*, vol. 33, pp. 3014–3027, 2025.

[15] Zhiqi Ai, Han Cheng, Shiyi Mu, Xinnuo Li, Yongjin Zhou, and Shugong Xu, "Effective User-Defined Keyword Spotting With Dual-Stage Matching, Multi-Modal Enrollment, and Continual Adaptation," *IEEE Transactions on Audio, Speech and Language Processing*, vol. 34, pp. 3214–3227, 2026.

[16] Jatin Agrawal, Bramhendra Koilakuntla, and Srikanth Konjeti, "Spot and Merge: A Hybrid Context Biasing Approach for Rare Word and Out of Vocabulary Recognition," in *Proceedings of Interspeech*, 2025, pp. 3319–3323.

[17] Kyubyong Park and Jongseok Kim, "g2pE: A simple python module for english grapheme-to-phoneme conversion," `https://github.com/Kyubyong/g2p`, 2019.

[18] Qwen Team, "Qwen3-ASR Technical Report," *arXiv preprint arXiv:2601.21337*, 2026.

[19] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur, "Librispeech: An ASR Corpus Based on Public Domain Audio Books," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5206–5210.

[20] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, Jian Wu, Long Zhou, Shuo Ren, Yanmin Qian, Yao Qian, Jian Wu, Michael Zeng, and Furu Wei, "WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.

[21] Zhuoyuan Yao, Di Wu, Xiong Wang, Binbin Zhang, Fan Yu, Chao Yang, Zhendong Peng, Xiaoyu Chen, Lei Xie, and Xin Lei, "WeNet: Production Oriented Streaming and Non-Streaming End-to-End Speech Recognition Toolkit," in *Proceedings of Interspeech*, 2021, pp. 4054–4058.